

Feedback Loops in the Agent Bestiary

Date: 2026-05-15, revised 2026-06-03, **verified and revised 2026-08-15, operational evidence + remaining fixes 2026-08-16** **Status:** Reference — describes the five adaptive feedback loops, their verified implementation state, what the first deploy actually demonstrated, and BayesOps Loop A (shipped through Phase 3). **Verified against:** `main @ 884daab3`, migrations through `210` .

Read §5 first if you want the short version. Every loop is wired, and §5 records which ones have been *observed turning* on real data versus which are still waiting for traffic. Those are different claims and this document keeps them apart.

How this document was verified

The 2026-06-03 revision described the design and asserted status. This revision walks each assertion back to code and records what was actually found. Three kinds of correction were needed, and the third is the one worth internalising:

- Stale locations.** Line numbers had drifted by hundreds of lines. This revision cites **file + symbol name** rather than `file:line` , because symbols survive refactors and line numbers do not. Where a line number appears it is indicative only.
- Stale status.** Several things marked "specified, not yet implemented" have shipped — most significantly all of BayesOps Phases 1–3.
- Declared ≠ dispatched.** The previous revision counted a tool named on an agent card as a working tool. It is not. A card can declare a tool name that has no dispatch arm in `ToolRegistry::execute ( src/agent_backend/tools_legacy.rs );` the name is advertised to the model, the model calls it, and it returns `Unknown tool: x` . The codebase now names this defect class a **phantom tool**. Two loops — 4 and 5 — were documented as closed through a phantom tool and were not. Both are now genuinely dispatched; see §7 for the corpus-wide ratchet that replaced the ad-hoc check, and for the 73 declarations still outstanding.
- Written ≠ readable.** A loop can complete every write correctly and still report nothing, because the surface that displays it queries a different table than the one the loop writes. Loop 1 was in this state: consolidation wrote entities and rules, and every knowledge surface read `ontology_snapshots` , which nothing on that path populates. Call this a **severed read path**. It is more dangerous than a phantom tool, because it presents as a *quiet* failure of the loop itself rather than as an error — the natural response is to go looking for a bug in the learning, which is working.

5. **Closed ≠ turning.** Added 2026-08-16, after the first deploy. Every hop having an executing call site does not mean the loop has moved. Loop 1's observation leg is closed and, at the moment of writing, has produced zero anomalies — not because it is broken but because **1,170 of 1,245 timeline entries sit at `persona_version = 1`**, which the drift monitor skips by design. Those are historical eval-run entries written before the stamping fix. The loop is correct, the corpus is not yet eligible. Reporting that as "closed" without the caveat would be the same overclaim this document was rewritten to remove. §5 separates the two.

A loop is only called closed here when every hop has an executing call site and the surface that reports it reads the tables the loop writes. It is only called *turning* when it has been observed to move on real data — see §5.

The deferred-work comment

One pattern produced three of the defects found on 2026-08-15, in three unrelated subsystems, and it is worth recognising on sight. Each is a comment asserting that some other component will finish the job:

Comment	Reality
<code>embedding: None</code> — <code>*"CEP seed entities have NULL embedding by design... the consolidation worker may later opportunistically embed"</code> *	It never did. 2,477 rows of curated reference knowledge, unreachable.
<code>embedding: None, // will be re-embedded by the consolidation worker</code>	It never did. Every HITL correction, unclusterable.
<code>*"Update the in-memory registry's ontology_stats so <code>enrich_with_kg_context</code> stops fast-pathing this agent"</code> *	Queried <code>kg_entities</code> , a table that has never existed. Error swallowed. Gate stayed shut for every agent.

The comment is what made each one survive review: it reads as a considered design decision, so the reader stops looking. None named the component that would do the work in a way anyone could check, and no test asserted the handoff. **A deferred-work comment is a claim about another component's behaviour, and should be treated as an untested assertion until a test pins it.**

The structural remedies, all cheap and all now in the codebase:

- `invalid_tool_declarations` (`src/agent_backend/tools_legacy.rs`) diffs declared tool names against dispatch arms. `no_curated_card_declares_a_phantom_tool` extends it to all of `agents/curated` as a ratchet — see §7.

- Payload assembly split into a pure function with count-vs-content assertions (`handlers::ontology::build_ontology_payload` and its tests) makes a severed read path fail in CI rather than in the UI. **Any handler that reports what a loop produced should be testable this way.** The invariant is one line: what the tables hold is what the payload reports.
- `src/schema_trust.rs` is the remedy for the whole class. Every defect in the tables above is ultimately a query naming a column or table that does not exist, with the error swallowed. `SCHEMA_COLUMNS` turns that into a CI failure. It works only for columns actually declared, which is why declaring the ones a loop depends on is part of closing it, not an afterthought — see the `detected_at` incident in §6.

Three smaller lessons from the same work

Worth recording because each cost real time and none is obvious:

1. **A dispatch arm is not enough to declare a tool.** Cards validate against `builtin_tools()`, a *separate* declarative list. A tool with an arm but no `BuiltinToolDef` runs correctly when called and yet cannot be saved through the API. `equity_analyst`'s nine `fmp_*` tools were in exactly that state since the agent shipped. Both halves are required, and they answer different questions — `dispatchable_tool_names()` for "will this call succeed", `platform_tool_names()` for "can this card be saved".
2. **The crate split is load-bearing.** `tools_legacy.rs` is in the `fermi` lib; `handlers/` is bin-only. **A tool cannot call a handler.** Anything both a route and a tool need must live in the library, which is why `compute_agent_calibration` moved to `fermi::calibration` rather than being duplicated.
3. **A warning nobody can action gets ignored, and then so does the linter.** `scripts/lint-migrations.sh` flagged `mig-200` for statements outside a DO block. Wrapping them did not silence it: the linter's strip regex only recognises the literal `$$` tag, and the migration used a named `$mig200$` tag. The fix was to use `$$` outside and named tags inside — not to dismiss the warning. Leaving a spurious warning in place is how a control earns the right to be ignored.

Status markers used below:

Marker	Meaning
✓ Wiring closed	Every hop verified with an executing call site
✓ Turning	Observed to move on production data — see §5
— Not yet turning	Wiring closed, waiting on traffic or data. Not a defect
◐ Partial	Closed on one path, broken or absent on another; the break is named
✗ Broken	Documented as working, verified as not working

The first three are the ones that matter. A loop can be wiring-closed and not turning for entirely legitimate reasons, and saying so is more useful than a single verdict that hides which.

Framing

A feedback loop is *negative* in the control-systems sense when the output signal is fed back to reduce the error between current behaviour and desired behaviour. Every loop described here does this: it measures a deviation from some target (coherence, accuracy, persona fidelity, team composition, routing quality), and the loop corrects toward it.

The word "negative" here does not mean harmful — it means stabilising and self-correcting. A thermostat is a negative feedback loop. So is evolution. So, if these loops work as designed, is a well-run Agent Bestiary composition.

What makes these loops *adaptive* rather than merely reactive is that the correction changes the internal state of the agent or composition permanently, not just its behaviour on the next turn. The agent that dreamed last night reasons differently today. That is adaptation.

What these loops do and do not change

A useful framing from the SIA paper (arXiv:2603.27766): *"Harness shapes how the agent searches; weight updates change what the model knows."*

All five loops described here are **harness-level changes**. They modify:

- What semantic rules, entities, and facts the agent's prompt is enriched with before each execution (Loop 1)
- Which anomalies a human reviewer sees and corrects (Loop 2)
- What coordination brief the agents read on the next turn (Loop 3)
- Who is in the composition (Loop 4)

- Which member the routing strategist selects (Loop 5)

None of these loops update model weights. They change the context, the configuration, and the routing — not the underlying model's parameters. This is the correct design for API-hosted models where weight updates are unavailable. It is also the correct design even when fine-tunable local models are available: harness changes are reversible, auditable, and human-gateable; weight updates are none of those things by default.

The quality ceiling question — whether harness-level accumulation of semantic rules reaches the same improvement ceiling as gradient descent — is empirically open. The architecture does not preclude weight updates for local models; the quality-weighted episode history produced by these loops is a direct prerequisite for any future fine-tuning path.

Loop A (BayesOps) is the one exception to the harness framing, and it is not a weight update either — see §4.

Two classes of eval signal

The loops consume two structurally different kinds of eval signal, and the difference matters:

LLM-judged signals — scores produced by evaluators that use an LLM to assess output quality (LlmJudge, Faithfulness, Sotopia, etc.). These are fast and domain-general but inherit LLM non-determinism. They require the coherence gate in Loop 2 because a sufficiently adversarial or confused judge could produce a correction that damages the agent's world model.

Hard-verified signals — scores produced by deterministic comparison against ground truth that resolves independently of the agent's output. Brier score on resolved forecasts (Loop 5) and `projection_accuracy` on real SOSA observations vs. prior cascade projections (Loop 1, Spec 20) are both hard-verified. The scoring step has no LLM in it. The ground truth (market resolution, physical batch measurement) does not know or care what the agent predicted.

Hard-verified signals are epistemically stronger: they cannot be gamed by an agent that learns to produce plausible-sounding outputs, and they do not require a coherence gate before propagating into memory. When a real cultivation batch yields 3.8 kg against a predicted 4.2 kg, that delta is a fact. The semantic rule it produces ("this model overestimates yield at high temperature") is grounded in physical reality, not in an LLM's judgment of output quality.

A third class has since been added, and it is not an eval signal at all:

Provenance signals — `stamp_invocation ( src/api_server.rs )` records *how an agent was asked and why it was chosen* as slugged, forgery-resistant episode tags (`route:{reason}` , `qsrc:*` , `ibind:*`). These carry no quality judgment. They exist so that the loops can separate

"the agent was sent the wrong question" from "the agent is bad at the job" — previously the same row. See `crates/fermi-console/src/negotiate.rs` for the contract that produces them.

The five loops

Loop 1 — Individual agent learning

Target: the agent should reason correctly about its domain, using what it has learned from past executions.

Signal: eval dimension scores written to `eval_signals` per evaluator per episode. Two classes of signal feed this loop:

LLM-judged dimensions — relevance, accuracy, completeness, persona_fidelity, and similar scores produced by the EvaluatorRegistry (LlmJudge, Faithfulness, Sotopia, etc.). Fast, domain-general, inherently noisy.

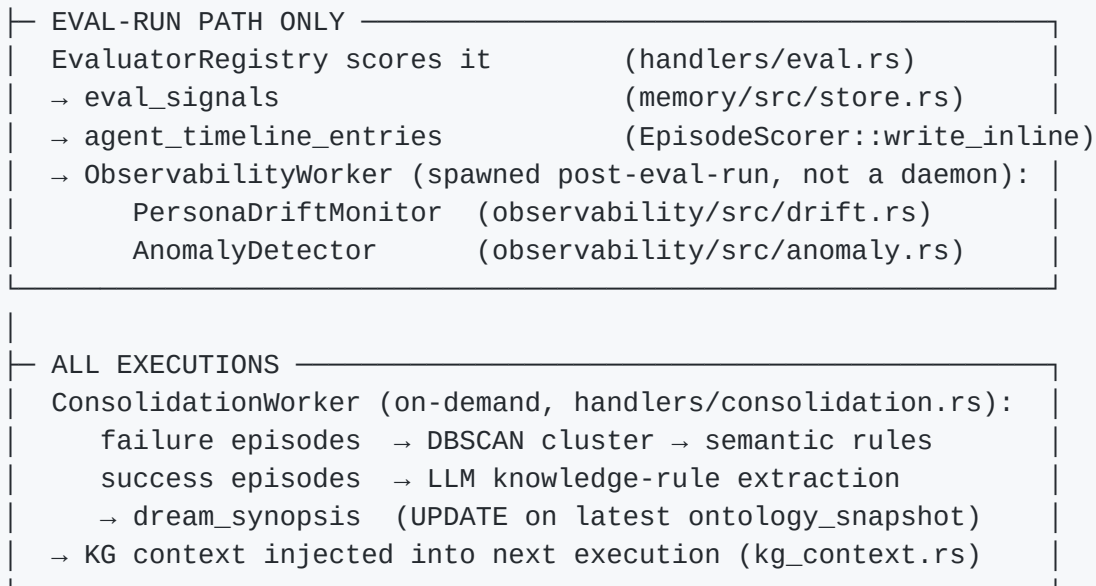
Hard-verified dimensions — scores computed by deterministic comparison against ground truth that resolves independently of the agent's output:

- `forecast_calibration` (Brier score on resolved `fermi_forecasts`) — Loop 5 feeds this back into Loop 1 for forecasting agents
- `projection_accuracy` (SOSA observation delta: $1 - |\text{predicted} - \text{actual}| / |\text{actual}|$) — introduced in Spec 20 for `simops_dynamics_runner` and `simops_cascade` agents; computed by `ProjectionScoringEvaluator` when a real batch measurement arrives against a prior synthetic projection

Hard-verified signals require no coherence gate before consolidation. They are facts about the physical world, not judgments about output quality.

Correction path (verified):

Agent executes → episode stored



For hard-verified signals (projection_accuracy):

```
Real SOSA observation ingested (agent_backend/simops_tools.rs)
→ ProjectionScoringEvaluator: find prior synthetic projection
  via CascadeProvenance.projection_id (crates/simops/src/cascade_v2.rs)
→ compute delta → write EvalSignal (dimension: "projection_accuracy")
→ same ConsolidationWorker path → semantic rules like:
  "kombucha_fermentation overestimates yield by ~15% when temp > 65°C"
→ injected into simops_dynamics_runner KG context on next execution
```

What changes: the agent's semantic memory — the rules, entities, and facts its system prompt is enriched with before each execution. The agent that has run 50 times on market analysis questions has accumulated domain-specific rules that make its 51st response qualitatively different from its first. For SimOps agents, hard-verified projection_accuracy scores produce physically grounded model-calibration rules with no LLM judgment in the scoring path.

Timescale: dreaming cycles for LLM-judged signals (hours to days). Hard-verified signals trigger consolidation as soon as a real observation arrives — potentially within the same session as the projection.

Status:  **Closed** — 2026-08-15. Both legs now run on live traffic:

- *Learning:* episode → consolidation → embedded knowledge → **retrieval into the next execution** (retrieval-gate note below).
- *Observation:* execution → timeline entry → drift + anomaly → HITL queue (`handlers::live_observability`).

Corrections to the previous revision:

- **The eval leg did not fire on live traffic — fixed 2026-08-15.** The 2026-06-03 revision said eval signals and timeline entries are written "inline, hot path". They were written inline *within the eval pipeline*: `EpisodeScorer::write_inline` had exactly one call site, inside `run_eval_cases`. Live executions stored an episode and injected KG context but produced no timeline entry, so `PersonaDriftMonitor` and `AnomalyDetector` never saw real traffic — and since Loop 2 is fed by anomalies, its queue was fed only by eval fixtures. See `handlers::live_observability` for the fix and the cost argument.
- **`create_snapshot` is not on the API path.** `agent-bestiary/ontology/src/snapshot.rs::create_snapshot` is called only from the standalone CLI (`agent-bestiary/consolidate/src/main.rs`). The API dreaming path writes `UPDATE ontology_snapshots SET dream_synopsis = ... WHERE snapshot_id = (latest for agent)`, which is a no-op for any agent whose snapshots were never created by the CLI.

The episode → consolidation → KG leg is genuinely closed and running, on all executions. That is the leg doing the actual learning.

Retrieval gate defect — found 2026-08-15, fixed. This was Loop 1's actual break. Writing knowledge does not close a loop; reading it back does. `enrich_with_kg_context` skipped injection entirely when `card.ontology_stats.entities == 0 && relationships == 0` — a field that nothing maintains:

- cards reconstructed from a DB row hardcoded `entities: 0` (`api_server.rs`)
- 31 of 100 curated card JSONs omit the block; every field is `#[serde(default)]`, so it deserialises to zero
- the sole updater counted `SELECT COUNT(*) FROM kg_entities` — **a table that has never existed**. The error was swallowed by `.ok().flatten().unwrap_or(0)`, so it wrote zero every cycle, while its own comment stated it existed "so `enrich_with_kg_context` stops fast-pathing this agent"

The gate was therefore closed for effectively every agent, permanently. Consolidation extracted entities and rules, embedded them, stored them correctly — and no execution ever read them back. An agent with a hundred learned rules behaved identically to one that had never dreamed. **Loop 1 was writing to a memory it could not consult.**

The gate now asks the knowledge tables directly (one indexed `EXISTS`, sub- millisecond, against the 300–800 ms embedding call it is deciding whether to spend). It also distinguishes a third state the old boolean could not express: *rows exist but none carry an embedding*. Retrieval is embedding-based on both the ANN and fallback paths, so such rows are unreachable — the agent has knowledge it structurally cannot recall. That state now logs a warning per execution

instead of being silently identical to "new agent". Semantics locked by

```
kg_context::gate_tests ; census in scripts/loop1_retrievability_census.sql .
```

The verified recall chain. Every hop below has an executing call site, and the last three are pinned by `kg_context::gate_tests` . This is what "Loop 1 is closed" now means concretely:

```
consolidation → entities / facts / semantic_rules (embedded at write)
  → retrievable_knowledge gate (queries the tables)
  → get_top_k_semantic_rules / get_top_k_entities_with_cep
                                     (pgvector ANN top-k;
                                     cep_% always injected)
  → build_kg_block_ann (renders rule + entity TEXT)
  → append_kg_block → card.system_prompt
  → ExecutionContext.agent_card
  → LlmExecutor::build_system_prompt
  → `system:` field of the provider request
```

`retrieved_knowledge_reaches_the_prompt_text` asserts the actual rule content and entity names appear in the block — not merely that a block was produced. A renderer that emitted headings and dropped the content would satisfy every count-based check while teaching the agent nothing. Per-execution the `kg_context_enrich` span now records `injected` , `rules` , `episodic_entities` , `cep_entities` and `block_chars` , so recall is auditable from logs rather than assumed; the previous span fired identically whether a hundred rules were recalled or none.

Seed facts are now embedded at write time — fixed. `seed_cep_entities` stored `embedding: None` on the reasoning that "the consolidation worker may later opportunistically embed `entity_name` if needed". Nothing ever did. That is survivable only for `cep_`-typed rows, which are injected unconditionally; everything else needs a vector to be reachable at all. Seeding now generates provenanced embeddings in one batch for facts that are actually new, so curated reference knowledge is retrievable, and it *scales* — always-injection is fine for a handful of constants but blows the context window at volume, whereas similarity retrieval returns the top-k that matter for the query. Embedding failure never blocks boot; the rows are written unembedded and the census reports them as stranded.

Seed idempotency — fixed. The guard was `existing.any(|e| e.entity_type.starts_with("cep_"))` while the loop it guarded wrote whatever `entity_type` the card declared. Cards whose seed facts are not `cep_`-prefixed could never trip it, so **every boot re-seeded the entire set**: 15 distinct facts stored as 2,475 rows, exactly 165 copies each, growing without bound. Idempotency is now per fact on `(entity_name, entity_type)` , so a card that gains a fact picks it up instead of being skipped wholesale. Cleanup:
`scripts/loop1_dedupe_seed_entities.sql .`

Ontology development — fixed. `create_snapshot` is now called on the API dreaming path (`handlers::consolidation::snapshot_ontology`), so the Mermaid diagram, git provenance and `evolution_commits` advance with each cycle instead of staying frozen at nothing. Failure is non-fatal and logged — consolidation's real output is already durable in the knowledge tables, and `MermaidGenerator::generate` legitimately refuses to draw an agent with no live entities (which a `?allow_degraded=true` run can produce). Push is hardcoded off on this path: `commit_ontology` is synchronous and its libgit2 push has no timeout, so an unreachable remote would block a tokio worker. This also repairs a latent no-op — the dream narrator's `UPDATE ontology_snapshots SET dream_synopsis` targeted a row nothing ever inserted, and now targets the cycle's snapshot by id rather than by a racy `ORDER BY version DESC LIMIT 1`.

Read-path defect — found 2026-08-15, fixed. The loop was succeeding and reporting zero. `handlers/ontology.rs::get_ontology`, which backs both the agent Knowledge tab and the `/agent/:id/ontology` viewer, read **only** `ontology_snapshots` — the one table on this path that nothing writes — and hardcoded `"entities": [] / "relationships": []` even when it found a row. Consolidation would report "5 rules, 4 entities", write them correctly to `entities / facts / semantic_rules`, and every knowledge surface would then show `Entities: 0 Relationships: 0`, with the Knowledge tab stuck on a literal ellipsis because the frontend gated its DOM write on a `stats` block the empty payload did not contain.

This is the inverse of the failure `dreaming_maturity` was built to catch: there the loop runs and learns nothing, here it learns and cannot show it. Both present identically to an operator. `get_ontology` now derives counts and graph content from the live knowledge tables; a snapshot contributes only its Mermaid diagram, git provenance and dream synopsis, and can no longer determine whether the graph appears. Locked by unit tests in `handlers/ontology.rs` — see §6.

Operational trap — recovery is order-dependent. The 2026-05-16 and 2026-06-22 extractor-less batch runs marked 1,035 episodes across 62 agents consolidated while learning nothing. `scripts/loop1_reset_unlearned_episodes.sql` recovers them, but gates on the agent having a *completely empty* ontology. Running a single successful consolidation on a damaged agent — the most natural way to investigate — gives it a non-empty ontology and excludes it from recovery permanently, while the rest of its history stays stranded. `fermi` hit exactly this. `scripts/loop1_reset_sterile_episodes.sql` recovers per-episode instead, using the `source_episodes` provenance arrays to identify episodes that were consumed and contributed to nothing, gated on positive evidence of a zero-yield job. **Run the dry run before re-dreaming a damaged agent, not after.**

Instrumentation added since: `/api/observatory/loops/dreaming/maturity` (`src/handlers/dreaming_maturity.rs`) classifies the "91 dreaming cycles, zero entities, zero

facts, zero rules" failure mode — a loop that runs and learns nothing. Check it before asserting this loop is working for any given agent.

See also: `docs/specs/20_SIMOPS_PROJECTION_SCORING.md` ;
`docs/specs/21_PERFORMANCE_SPEC_2026-06-05.md` for the HNSW/ANN retrieval and
`spawn_blocking` DBSCAN rework (behaviour-preserving).

Loop 2 — Behavioral correction via HITL

Target: the agent's behaviour should align with human judgment, especially on high-stakes or anomalous cases.

Signal: human reviewer decisions — Approve, Relabel, Intervene — applied to anomaly events surfaced by Loop 1.

Correction path (verified):

```
Anomaly detected (Drift, RollingConflict, Rupture, Safety) → anomaly_events
  → HITL review queue (/observatory/hitl; handlers/observatory.rs)
  → Reviewer acts: Intervene
  → InterventionEncoder: validate, stamp authority_weight=1.0
                        (coherence-gate/src/encoder.rs)
  → CoherenceGate: check  $\Gamma(C)$  against DEFAULT_GATE_THRESHOLD = 0.5
                  (coherence-gate/src/gate.rs)
    · AgentWide scope → blocking
    · Episode / Dyad scope → "settler mode": advisory, never blocks
  → AgentWide only: second-reviewer consensus required
    POST /api/observatory/hitl/consensus/:request_id
    different user enforced (handlers/observatory.rs)
  → TwoWriteMemory (coherence-gate/src/two_write.rs):
    Write 1: synthetic episode (SyntheticCorrection, authority=1.0)
    Write 2: episode_corrections (audit trail, coherence_check,
                                   minimum_update_set)
    AgentWide: bump_persona_version()
  → Synthetic episode enters Loop 1 → consolidated
  → New persona_version creates new drift baseline
```

What changes: the agent's persona — its effective belief system as encoded across its episodic memory. An agent-wide intervention marks a version boundary: before and after the correction, the agent's behaviour is measurably different (drift monitor will detect this). The correction is preserved in the immutable audit trail.

Safeguards (verified, with two corrections):

- The gate blocks only at `AgentWide` scope. `Episode` and `Dyad` corrections run advisory — the previous revision's unqualified "blocks corrections that would create incoherence" overstates this. It is a deliberate design choice (`gate.rs` settler-mode comment), not an oversight.
- The second-reviewer requirement for agent-wide corrections is real and enforced by user identity, not merely documented.

Correction — the synthetic episode could not propagate. Fixed 2026-08-15. The previous revision said the correction "enters Loop 1 → consolidated at HumanAuthority weight". Neither half was true, for two independent reasons:

1. **It was written unembedded.** `two_write.rs` passed `embedding: None` with the note *"will be re-embedded by the consolidation worker"*. *It never was — the worker embeds the rules and entities it *extracts*, never the episodes it reads. Every episode query on the clustering path filters `embedding IS NOT NULL`, so the correction was invisible to DBSCAN.
2. **authority_weight was never read.** Rule extraction did `.take(30)` straight off `get_unconsolidated_episodes`, which returns `ORDER BY timestamp_ref DESC`. An agent that had run thirty times since the correction dropped it from extraction entirely, silently.

Together: a human said "this is wrong, here is the right answer", it passed the coherence gate, it was signed off by two independent reviewers for agent-wide scope — and whether it ever reached the agent depended on how busy that agent had been since. **The single highest-authority signal in the system was the one that could not propagate.**

Both fixed. `TwoWriteMemory::with_embedder` embeds the correction on `query` at write time, matching how live executions embed episodes; embedding failure logs and still stores, because the audit trail is load-bearing and retrievability can be backfilled while a dropped human decision cannot. `rank_success_episodes_by_authority` orders by authority before applying the budget, stably, so ordinary consolidation is not reshuffled. Locked by `consolidation::authority_tests` — including `human_corrections_survive_the_extraction_budget`, which places the correction as the *oldest* of 41 episodes and asserts it is extracted first.

Timescale: human-initiated, but the effect propagates in the next dreaming cycle.

Status:  **Wiring closed — 2026-08-15. Not yet turning.** The HITL mechanism (queue, encoder, gate, two-write, consensus, audit trail) was already closed and verified. The *propagation* path — correction → embedded episode → clustered → semantic rule → injected into the agent → changed behaviour — is now closed for the first time, and rides on the same retrieval chain Loop 1 uses.

One upstream dependency remains, and it is not Loop 2's own: live traffic produces no eval signals and therefore no anomalies (Loop 1 status, break #5), so the HITL queue is fed only by eval runs. Loop 2 now works correctly on everything it receives; what limits it is how little reaches it.

Loop 3 — Workspace coherence

Target: a workspace's multi-agent conversation should produce coherent, evidence-grounded outputs without suppressing productive disagreement.

Signal: $\Gamma(C)$ — the global coherence score from TEC settling — plus per-principle scores (P1 Symmetry, P2 Explanation, P3 Analogy, P4 DataPriority, P5 Contradiction, P6 Competition, P7 Acceptability; `coherence-core/src/principles.rs`) that distinguish productive incoherence (low P6 with high P4) from destructive incoherence (low P2, low P7).

Correction path (inner — per session), verified:

```
Workspace messages accumulate
  → Auto-coherence evaluation every N messages
    (COHERENCE_AUTO_EVAL_INTERVAL, default 10; workspace/messages.rs)
  → ConversationObserver → SettlingEngine::with_defaults().settle
  →  $\Gamma(C)$  + principle scores → coherence_evaluations row
  → posts a `coherence_update` system message
  → STOPS HERE. Does not invoke the strategist.

  → User triggers via Coherence shelf at Recommendations or Dream Notes tier
    (workspace/coherence.rs)
  → cohere_and_coordinate executed via registry.execute_agent(...)
  → NO ToolContext is constructed, therefore NO tools are available
  → Stage 0 (intention map) and Stage 3 (write brief) cannot execute
```

Correction path (outer — across sessions):

```
cohere_and_coordinate accumulates session episodes in its own memory
  → Composition Dreaming (POST /api/workspaces/:id/composition/dream)
    → posts an @cohere_and_coordinate [COMPOSITION DREAMING — TENSION AUDIT]
      message (handlers/composition.rs), charges 5 credits
    → Stage 4 / valence-homophily threshold (spread < 0.25) exists as
      PROMPT TEXT ONLY. No Rust computes arousal or valence spread.
  → propose_composition_change → PHANTOM TOOL (see Loop 4)
```

What changes — and the mechanism the earlier revisions got wrong.

Both previous revisions described Loop 3's correction as *"agents read the coordination brief in their next turn context"*. That was never the design and could not have worked: a brief is a file, and nothing reads it — workspace auto-injection loads only `context/`, and consolidation reads `episodes`.

The actual mechanism is a **cascade into member memory**. The strategist observes how each member behaved and writes that observation *into that member's episodic memory* via `record_coordination_observation`. The member picks it up on its next dreaming cycle, distils it into a semantic rule, and carries it into every later execution through KG injection.

```
Strategist observes member behaviour in a session
→ record_coordination_observation(agent_id, observation)
→ episode in THAT MEMBER's memory
    provenance = coordinator_observation    (not a run – mig-200)
    authority_weight = 0.6                  (above ordinary, below human)
    embedded at write time                  (or it cannot cluster)
    consolidated = false                    (so dreaming picks it up)
→ member's ConsolidationWorker → semantic rule
→ member's KG context on every subsequent execution
```

This is Loop 3 → Loop 1 cascade, and it is what makes Loop 3 *adaptive* rather than advisory: the correction changes the agent's memory permanently, not the direction of one conversation. The brief remains, but as a document for the humans reading along — it is not the mechanism.

- Inner loop: the coherence update message steers the current conversation; the cascade changes what each member knows next time.
- Outer loop: composition proposals — see Loop 4.

Timescale: inner loop runs within the session (minutes). Outer loop requires accumulated session history and human approval (days to weeks).

Status:  **Closed — 2026-08-15.** Three defects fixed:

0. **The cascade had no mechanism.** Stage 4 instructed the agent to "write a context episode via `write_workspace_file` to `_coordination/cascade/<agent_name>.md`", which reads like the right thing and does nothing — dreaming reads `episodes`, not workspace git. Replaced with `record_coordination_observation`, which writes a real episode into the member's memory. Because this writes into *another agent's* memory it is the one tool where a missing check is a memory-poisoning primitive, so it is gated twice: the caller must be the workspace's registered `coordination_strategist_id`, and the target must be a current member of that workspace.

And the two that blocked it from running at all:

1. **Auto-eval does not invoke the strategist.** It stores the evaluation and posts a system message. Strategist invocation happens only on the user-triggered shelf path. The previous revision's `OR` in the diagram was actually an `only`.
2. **The strategist ran without tools — fixed.** The shelf called `registry.execute_agent` directly, which builds no `ToolContext`, so the strategist had no tools at all. Its card declares a four-stage protocol that is almost entirely tool calls, and none of them could execute — the shelf returned prose describing work the agent had not done. Now routed through `ToolAwareExecutor` with a full `ToolContext`.
3. **The brief was unreadable anyway.** Auto-injected workspace context loads only files under `context/`; `_coordination/brief.md` is outside that prefix. This no longer matters for closure, because the brief is not the mechanism — but it is why treating it as one never worked.

Also verified working throughout: $\Gamma(C)$ measurement, per-principle scoring, the auto-eval cadence, and the `coherence_update` message.

Coherence signal semantics changed after the previous revision — see `754edd39` (relevance gating, uptake-based Symmetry, principle checks that can actually fire) and `5a9f925c` (dyads / companion loop). The P1–P7 description above reflects the current semantics.

Loop 4 — Composition evolution (tune-team RSI)

Target: the composition's team structure should improve over time to reduce chronic coordination failures and redundant membership.

Signal — this is what changed most since 2026-06-03. The previous revision described the signal as "recurring patterns in `cohere_and_coordinate`'s consolidated memory". That path never produced a single proposal. The module that replaced it says so directly:

`composition_versions` has had an accept/reject flow since mig-113, and the dashboard has always had a card for it. It permanently read "no pending evolution proposals" because **nothing ever generated one**. The loop was structurally complete and empty: a mechanism with no signal feeding it. — `src/handlers/composition_evolution.rs`

The signal now exists, and it is quantitative: **exact Shapley attribution**

(`src/attribution/counterfactual.rs`, migrations 187–188) computes per resolved forecast:

- `forecast_agent_credit` — each agent's marginal contribution ϕ
- `forecast_agent_interactions` — whether each *pair* is synergistic or redundant

The pairwise interaction index is the load-bearing part. Marginal credit alone cannot answer "who should be on this team" — an agent can be individually valuable yet wholly redundant with a cheaper one.

Correction path (verified):

```
Resolved forecast → exact Shapley decomposition (src/attribution/)
→ forecast_agent_credit ( $\phi$ ) + forecast_agent_interactions
→ GET /api/workspaces/:id/composition/suggestions
  (composition_suggestions_handler)
  · candidates: mean_credit < 0 AND n_forecasts >= 5
    (MIN_FORECASTS_FOR_PROPOSAL – suppressed below this, deliberately)
  · every proposal carries the sample size it rests on
→ POST /api/workspaces/:id/composition/suggestions/materialise
→ composition_versions row (accepted_by IS NULL AND rejected_by IS NULL)
→ Owner: Accept → memory/src/store.rs apply path ⚠ SEE DEFECT BELOW
  Reject + note → episode in strategist memory
    (Provenance::HumanCorrected, authority_weight 1.0,
     tags: composition_rejection / dreaming_material)
→ Rejection feeds back into Loop 1 for the strategist ✅ verified
```

Why proposals are generated but not applied — quoting the module, because the reasoning is the design: attribution measures contribution *through the current model*, so a negative ϕ can mean a weak agent, a mis-specified driver exponent, or a genuinely predictive driver that is currently mis-weighted. Automatic pruning would let a modelling error silently strip the roster.

Status: ✅ Wiring closed — 2026-08-15. Not yet turning.

Two defects, both fixed:

1. **propose_composition_change was a phantom tool.** Declared with a full `input_schema` in `agents/curated/cohere_and_coordinate/agent_card.json`, with the composition-dreaming prompt instructing the agent to call it (`handlers/composition.rs`) — and no dispatch arm. Card tools carrying a schema are advertised verbatim, so the model *did* call it and received `Unknown tool: propose_composition_change`. **This is why the strategist path produced zero proposals for its entire existence.** It now has both a dispatch arm and a `BuiltinToolDef`, and writes a pending `composition_versions` row. It deliberately does not accept `member_agent_ids` — the card is explicit that naming the replacement is the owner's decision.
2. **The accept path wrote to a column that does not exist.** `agent-bestiary/memory/src/store.rs` ran `UPDATE teams SET member_weights = $1` **twice** — once bound to the roster array, once to the weights. `teams` has neither `member_agent_ids` nor `member_weights`; only `composition_versions` does (mig-113).

So accepting any version carrying members would error, and the 2026-06-03 claim that accept updates `teams.member_agent_ids` was never true at any point. The accept path now reconciles `workspace_agents` (mig-015) transactionally, additive-first, with the coordination strategist exempt from eviction — losing the agent that authors composition proposals as a side effect of accepting one of its own proposals would be a surprising way to lose it.

Why it is not turning. Both generators work and the accept path applies. The blocker is upstream of the loop entirely: **127 workspaces, none of which has a composition identity** — no mission, no strategist — so there is nothing to version. That is an onboarding gap, not a loop defect (§5).

Timescale: weeks to months. `MIN_FORECASTS_FOR_PROPOSAL = 5` is a floor, not a target; the loop is young and a confident proposal derived from two correlated forecasts would be worse than no proposal.

Important distinction from Loop 3: Loop 3's inner iteration changes conversation direction (fast, within-session). Loop 4 changes team composition (slow, across-sessions). They operate at different timescales and different levels of the system.

See also: `docs/architecture/COMBINATORIAL_CREDIT_ASSIGNMENT.md`.

Loop 5 — Calibration and routing accuracy

Target: the platform's agents should become more calibrated over time; the MoE routing strategist should learn which members are genuinely accurate on which sub-domains.

Two signal paths feed this loop:

5a — Forecast calibration (Brier score)

- Signal: Brier score when `fermi_forecasts` resolve against actual outcomes. Computed by `BrierEvaluator` (`handlers/eval_brier.rs`, `BrierLookupSqlx`), written to `eval_signals.dimension = "forecast_calibration"`.
- Timescale: months. Requires sufficient resolved forecasts to establish calibration curves.
- Ground truth source: market resolution, event outcomes — independent of the agent's prediction.

5b — SimOps projection accuracy

- Signal: `projection_accuracy` score when real SOSA observations arrive against prior cascade projections. Computed by `ProjectionScoringEvaluator`

```
( handlers/eval_projection.rs , ProjectionLookupSqlx ), written to  
eval_signals.dimension = "projection_accuracy" .
```

- Timescale: days to weeks, depending on batch cycle time. Ground truth arrives with every completed cultivation run — far faster than forecast resolution.
- Ground truth source: physical batch measurement — the batch does not know what was predicted.
- **Key difference from 5a:** this signal is available for SimOps agents even when no `fermi_forecasts` exist. It feeds Loop 1 directly (semantic rules about model calibration) and Loop 5 routing (which dynamics model to select for which process conditions).

Verified state of the signal paths:

5a – Forecast calibration:

```
Agent executes forecast question  
→ BrierEvaluator reads fermi_forecasts filtered on agents_used ✓  
→ Computes 1 - brier_score → forecast_calibration dimension ✓  
→ Written to eval_signals ✓  
→ resolve_forecast_handler (handlers/forecasts.rs) spawns:  
    find episodes tagged `moe_routing_decision` in last 7 days  
    → UPDATE episodes SET context with  
        outcome_quality (= 1 - brier.clamp(0,1)), outcome_source  
        ("brier_forecast"), outcome_brier_score, outcome_annotated_at ✓  
→ GET /api/agents/:id/calibration serves it ✓  
→ moe_router_strategist Stage 0 calls get_agent_calibration ✓  
    (dispatch arm + BuiltinToolDef; shares fermi::calibration with  
    the route, so the two cannot drift)
```

5b – Projection accuracy:

```
Cascade projection runs → synthetic SOSA observation written,  
projection_id stamped via CascadeProvenance  
(crates/simops/src/cascade_v2.rs, agent_backend/simops_tools.rs) ✓  
→ Real batch completes → operator enters SOSA observation  
→ ProjectionScoringEvaluator: match projection → compute delta ✓  
    (registered in EvaluatorRegistry, handlers/eval.rs)  
→ EvalSignal (projection_accuracy) → ConsolidationWorker ✓  
→ surfaced in the calibration response as projection_accuracy_mean ✓  
→ router reads it through the same tool ✓  
Migration 130 deployed long ago (repo is at 210).
```

The break, as it stood on 2026-08-15 (fixed 2026-08-16). The previous revision marked Loop 5 closed. Every *producer-side* claim in it holds: the evaluators are wired, the annotation fires on resolution, the endpoint is live and returns all five documented fields. But the consumer cannot read it. `agents/curated/moe_router_strategist/agent_card.json` declares `get_agent_calibration`; `ToolRegistry::execute` has no arm for it. The only implementation is the HTTP route `src/api_server.rs` →

`handlers::agents::get_agent_calibration_handler` . Stage 0 calls the tool, gets `Unknown tool` , and the card's own cold-start fallback ("calibration data not yet available") makes the broken wire look like sparse data. `debate_strategist` and `vote_strategist` carry the same declaration.

Also worth correcting: the doc's headline field, `calibration_score` , is the one the handler's own doc-comment warns against — `"Gate 'is this loop closed?' on skill, not on calibration_score , which is inflated by outcome-skewed question sets."` Use `brier_skill_score` , which the previous revision omitted entirely.

Both of the 2026-08-15 gaps are closed. The `get_agent_calibration` dispatch arm exists and shares `fermi::calibration::compute_agent_calibration` with the HTTP route, so the two cannot drift. The phantom-tool check now scans all of `agents/curated` as a ratchet rather than four hardcoded weather agents; the corpus scan that estimated 27 offenders in fact found 92, now 73 (§7).

What remains for Loop 5 is data, not wiring. `agents_used` carries entries that resolve to no agent, so the mechanism probe reports `WIRING BROKEN` and declines to certify the score — correctly. See §5.

Timescale: 5a: months (forecast resolution cadence). 5b: days to weeks (batch cycle cadence).

Status:

- 5a:  Wiring closed — signal collection, outcome annotation, endpoint and router read path all verified. **But the mechanism probe reports `WIRING BROKEN` on data grounds** (§5): 7 scored forecasts have an empty roster and 6 roster entries name no agent, so those Brier scores can never reach an agent's calibration. Read `brier_skill_score` , not `calibration_score` .
- 5b:  Wiring closed — full evaluator chain deployed and the router can read it; awaiting a first real SOSA observation cycle for operational evidence.

See also: `docs/specs/20_SIMOPS_PROJECTION_SCORING.md` for 5b implementation detail.

2. The hierarchy

The five loops operate at different timescales and different system levels:

Timescale 16)	Loop	Wiring	Observed turning? (2026-08-
Hours rows	1a. Individual learning	✓	✓ 8 eval runs, 23 ontology
Hours	1b. Projection accuracy	✓	– no real SOSA cycle yet
Hours deploy	1c. Live observation	✓	– no live traffic since
Days 1c)	2. HITL correction	✓	– queue empty (depends on
Session	3a. Coherence (inner)	✓	✓ 6 evaluations, $\Gamma(C)$ 0.97
Weeks	3b. Coherence (outer)	✓	– needs session history
Months composition	4. Composition evolution	✓	– no workspace has a
Days-weeks	5b. Projection calibration	✓	– awaiting first observation
Months+	5a. Brier calibration	✓	✓ MECHANISM SOUND, 9/9
Offline resolution	A. BayesOps – parameter fit	✓	✓ refits on workspace
built	(feeds Loop B / FPL executor)		Phases 1–3; Phase 4 not

Two columns, two claims. *Wiring* means every hop has an executing call site — the standard this document held itself to on 2026-08-15. *Observed turning* means it has moved on production data, which is the standard §5 holds it to. Conflating them is how the 2026-06-03 revision came to report closed loops that had never run.

The nesting is real rather than aspirational, and now runs in both directions: Loop 2 → Loop 1 (corrections become embedded episodes that survive the extraction budget), Loop 3 → Loop 1 (coordination observations become semantic rules in member memory), Loop 5 → Loop 4 (Shapley attribution generates composition proposals), Loop 1 → Loop 2 (live traffic produces anomalies that reach the HITL queue).

Loop 3's outer iteration was *supposed* to feed Loop 4 and never did — the `propose_composition_change` phantom tool meant the tension audit could conclude "the team should change" and end in `Unknown tool`. The Shapley path in `handlers::composition_evolution` replaced it as the primary generator; the tool now works as the qualitative second path.

Closure means the mechanism runs end to end with an executing call site at every hop. It does not mean the loop has been observed to turn. §5 records which have, on real data, and which are still waiting.

3. Loop 5 — Closure Status (revised 2026-08-16)

The four steps of the original plan, re-verified:

Step	Status	
Bootstrap calibration data (backtest seed)	✓ BrierLookupSqlx wired to fermi_forecasts	s
GET /api/agents/:id/calibration endpoint	✓ Live — calibration_score , brier_skill_score , trend , domain_calibration , projection_accuracy_mean , model_accuracy	rc h
get_agent_calibration tool on moe_router_strategist	✓ Dispatch arm + BuiltinToolDef ; shares fermi::calibration::compute_agent_calibration with the route	s s
Routing episode outcome annotation	✓ Fires on forecast resolution	s

All four steps are closed. Loop 5's remaining problem is not wiring — it is data. See §5: the mechanism probe reports WIRING BROKEN because fermi_forecasts.agents_used contains entries that resolve to no agent, so scored forecasts exist whose Brier can never reach an agent's calibration.

New since 2026-06-03: routing moved to a measured substrate

The previous revision modelled Loop 5 as endpoint + episode annotation. Three changes since have made routing itself measurable:

- a) Route provenance on every episode (7b768a08). stamp_invocation (src/api_server.rs) writes caller-supplied invocation records as slugged episode tags — route:{reason} , route:fallback , qsrc:* , ibind:* . Values are slugged (≤64 chars, restricted charset) so a caller cannot forge status:success . Contract in crates/fermi-console/src/negotiate.rs ; bind_input separately checks whether the agent even declares a free-text input.
- b) Agents declare the domains they serve (67066e4a). AgentContract gained domains and domains_explicit , parsed from metadata.domains with a metadata.tags fallback. RouteReason::DeclaredSpecialist is evaluated ahead of the hardcoded table in routing.rs . An explicitly empty domains: [] is a meaningful opt-out.

This fixed a live failure worth recording: `routing::domain_specialist` is a `match` over four domains that omitted `climate`, so every weather driver fell through to `macro_forecaster` — London 32 °C returned 0.3 % against a 13.3 % ensemble truth, and the divergence panel presented the gap as a trading signal. A new domain now needs a card edit, not a release.

c) Routing decisions joined to realised outcomes in SQL

(`migrations/193_route_provenance_outcomes.sql`), five views:

View	Answers
<code>route_outcomes</code>	Per-run join: route provenance → Brier + signed Shapley credit
<code>route_reason_performance</code>	Does a routing reason beat <code>default</code> , per domain?
<code>domain_agent_ranking</code>	Measured replacement for <code>domain_specialist()</code>
<code>router_override_scorecard</code>	Was overruling Fermi's suggestion right?
<code>declaration_quality_outcomes</code>	Do richer contracts produce better outcomes?

Headline metric is `avg_shapley` — per-agent and signed, so unconfounded by forecast difficulty in the way a raw Brier average is.

Known weakness, carried deliberately: `episodes` and `forecast_agent_claims` share no correlation id, so `route_outcomes` joins heuristically on `(agent_id, driver)` within -2 min/+10 min. It can **miss** when an agent is invoked twice on the same driver in-window; it **cannot mis-attribute** across agents or drivers. The fix — stamping `episode_id` onto the claim row in the multiplier hook — is deferred.

The cold-start progression

Holds as originally described, and the substrate for stage three now exists:

- **Month 0–2:** routing on `accepts / produces / skills / domains` declarations (semantic matching)
- **Month 2–4:** routing weighted by historical accuracy as forecasts resolve
- **Month 4+:** routing as a calibrated probabilistic classifier, with `domain_agent_ranking` replacing the hardcoded table

The value of the system increases monotonically with data. The architecture degrades gracefully to semantic matching at low data volume — that is the right design.

4. BayesOps — Loop A: Parameter Fitting (shipped)

Status: Phases 1–3 shipped 2026-06-16. Phase 4 not built. Phase 5 shipped in a different shape than specified. The previous revision's "specified, not yet implemented" and the "zero implementation" note in `docs/specs/14_BAYESOPS_SPEC.md §12` are both stale; `docs/fermi/BAYESOPS_CONTRACT.md` and `docs/specs/23_BAYESOPS_WORLD_CUP_DEMO.md` are current.

What Loop A is and why it is not Loop 1–5

Loops 1–5 are all **harness-level changes**: they modify what context agents receive, how they are routed, and how their compositions are structured. They operate over agent episodes and produce semantic rules, coordination briefs, and routing weights.

Loop A is different in kind. It operates **upstream of Loop B** (the FPL Monte Carlo executor) and produces something the loops do not: **the distribution parameters themselves**.

```
Loop A (BayesOps – offline, per dataset):
  Historical observations
    → fit posterior distribution
    → FittedDistribution: Beta(9.4, 13.6) or Normal(4.8, 0.7)
    → written into FPL Driver as distribution parameters

Loop B (FPL executor – online, per forecast question):
  Driver yield: Beta(9.4, 13.6) ← from Loop A, or from a human
    → Monte Carlo simulation (10,000 samples)
    → ExecutionResults: mean, p5, p95, Sobol indices
```

Loop B is entirely unchanged by BayesOps. The seam between Loop A and Loop B is the `Distribution` type in the FPL AST — `Beta`, `Normal`, `Lognormal`, `Triangular` — which already exists. Loop A produces those parameters from data rather than from human elicitation.

Phase status (verified)

Phase	Deliverable	Status	Dependencies
1	<code>crates/posterior</code> marginal fitting	✓ Shipped	<code>fit_marginal</code> , <code>FittedDistribution</code> , <code>FitMetadata</code> , <code>DataQuality</code> : in <code>beta.rs</code> / <code>normal.rs</code> / <code>lognormal</code> / <code>bootstrap_ci</code>
2	<code>crates/posterior-reg</code> HMC conditional fitting	✓ Shipped, one model	<code>fit_conditional</code> , <code>ConditionalRegressionConfig</code> , NUTS 4- (<code>sampler.rs</code>), R-hat/ESS (<code>diagnostics.rs</code>), <code>LinearNormal</code> exists, so the "outliers injected" gate is unnecessary
3	Four what-if query methods	✓ Shipped	<code>whatif.rs</code> : <code>predict</code> , <code>input_sensitivity</code> , <code>compare_scenarios</code> , <code>prob_exceeds</code> Gap: <code>HeteroscedasticNormal</code>
4	SimOps <code>PredictorEngine::Conditional</code> behind a <code>bayesian</code> feature	✗ Not found	No posterior dependency in <code>crates/posterior</code> / <code>PredictorEngine</code> , no <code>bayesian</code> feature
5	<code>data_driven()</code> in <code>parser/AST/executor</code> + posterior store + refit trigger	• Superseded in shape	No <code>data_driven()</code> anywhere <code>learnable: true + feeds_forward</code> (<code>src/parser.rs</code>), resolved in <code>(fitted_distribution_for ... PriorFallback, Static}</code> , <code>log_execution_results</code> / <code>ExecutionResults.learnable</code>

Tests: `cargo test -p posterior -p posterior-reg` — 62 + 39 unit, 6 end-to-end (`recovers_known_linear_posterior`, `prob_exceeds_is_calibrated`, `compare_scenarios_identifies_winner`, `optimise_for_target_finds_higher_x`, and two more), 2 doc-tests. All pass.

Surfaces: all seven operations exposed over both MCP (`src/bin/agent-mcp-server.rs`: `fermi_fit_marginal`, `fermi_fit_conditional`, `fermi_predict`, `fermi_input_sensitivity`, `fermi_compare_scenarios`, `fermi_prob_exceeds`, `fermi_optimise_for_target`) and HTTP (`src/handlers/bayesops.rs`, ~900 lines, plus posterior cache list/evict, workspace state, pending accept/reject, manual refit).

What actually feeds Loop A today

Not SOSA. There is no wiring from `sosa_observations` into `fit_marginal`. The live feed is **workspace resolutions** (Spec 23, R-1):

```
Workspace resolution committed
→ post-commit hook (handlers/workspace/resolution.rs)
→ refit_workspace (handlers/workspace/refit.rs)
  collect: feeds_from.source == "upstream_resolutions"
          → registered Extractor (crates/posterior/src/extractors.rs)
          + workspace_outputs
  fit:      fit_marginal
  gate:     Monte Carlo impact gate
  apply:    auto-accept → write_fitted_params (params.<driver>_fitted)
           otherwise   → stage a pending row for human review
→ persisted: bayesops_posterior_snapshots / bayesops_pending_fits
             (migrations/148_bayesops_refit_ledger.sql)
```

Manual trigger: `refit_workspace_handler`. Conditional posteriors are held in a `DashMap` cache only — persistent conditional storage remains unbuilt.
`harness_snapshots.bayesops_params` is still written null in `handlers/forecasts.rs`, though `forecast_benchmark.rs` accepts and hashes the column.

How Loop A relates to Loops 1 and 5

Extends Loop 1 (agent learning): Loops 1 and 5 accumulate `projection_accuracy` eval signals when real batches resolve against cascade projections (Spec 20). Those signals feed semantic rules into the agent's KG context — harness-level changes that tell the agent *which model is unreliable under which conditions*. Loop A adds the complementary capability: given that an agent knows which model to use, BayesOps provides *calibrated distribution parameters for what that model predicts*.

	Mechanism	Output	Level
Loop 1 / Spec 20	EvalSignal → consolidation → semantic rule	"Use bc_optimization at 30 °C, not kombucha_fermentation"	Harness
Loop A / BayesOps	Observation history → posterior fit → <code>Beta(α, β)</code>	"At 30 °C, yield follows <code>Normal(4.8, 0.6)</code> based on 40 real runs"	Distribution parameters

Together: Loop 1 tells the agent *what to run*; Loop A tells the FPL model *how to parameterise it*.

Extends Loop 5 (calibration and routing): the `ConditionalPosterior` produced by `posterior-reg` generates input sensitivity indices, scenario comparisons, and probability-at-threshold queries ($P(\text{yield} \geq 5.5 \text{ kg} \mid \text{lighting} = 135)$). These are scored by the same Brier/projection_accuracy infrastructure Loop 5 already uses — the fitted model's predictions resolve against real outcomes, feeding evidence about which BayesOps model variant is most accurate for which conditions. With only `LinearNormal` implemented there is currently one variant to choose between, so this is capability-in-waiting rather than an operating loop.

Remaining Loop A work

1. Additional regression models (`StudentT` , `HeteroscedasticNormal` , `NonlinearNormal`) — without them the improvement ladder in `improvement.rs` is a one-element walk and model selection cannot be validated
2. Phase 4: SimOps `PredictorEngine::Conditional`
3. Persistent storage for conditional posteriors (currently cache-only)
4. A SOSA-history feed into `fit_marginal` , which is what the original spec framed the loop around
5. Populate `harness_snapshots.bayesops_params`

See `docs/specs/14_BAYESOPS_SPEC.md §12` (sequencing — note the phase-status lines there are stale), `docs/fermi/BAYESOPS_CONTRACT.md` , and `docs/specs/23_BAYESOPS_WORLD_CUP_DEMO.md` .

5. Operational evidence — first deploy, 2026-08-16

Everything above is a claim about wiring. This section records what has actually been **observed to move** against the production database, and what has not. Measured with `scripts/loop1_retrievability_census.sql` and the loop-health panel, which derive independently of the code paths they assess.

Confirmed turning

Seed embedding and idempotency — the clearest result. Boot re-seeded the three damaged agents, and the numbers are exact:

Agent	Before	After	Embedded
football_institution_agent	1,155 unreachable	7	7/7
fixture_context_agent	660 unreachable	4	4/4
macro_data_agent	660 unreachable	4	4/4

duplicate_seed_facts: 0 . Previously every boot appended another copy of the whole set — 15 distinct facts had become 2,475 rows at 165 copies each. This boot added exactly one of each. **2,477 unreachable rows became 15 retrievable facts.** Platform-wide: 1,096 embedded, 107 CEP seeds, 26 unreachable — and all 26 are test fixtures.

The observability sweeper ran. 93 agents carry a checkpoint; backlog went **14 → 0**. That is the corrected last_scan_completed_at predicate executing against the real schema, which is the detail the first draft got wrong.

Loop 1a, measured independently. The loop-health panel, which derives from eval_runs ✕ eval_signals and consolidation_jobs ✕ entities/facts/semantic_rules rather than from anything asserted here: `""8 eval runs, 3 dimensions scored, and 3 dreaming cycles wrote back 23 ontology rows. Both halves turning."`

Loop 3a, measured independently. `""6 evaluations across 1/127 workspaces, Γ(C) mean 0.97."`

mig-200, verified behaviourally rather than by inspection, with a control:

executions_before	266	
after a coordinator_observation	266	← excluded, as designed
after an ordinary auto_pass	267	← real runs still counted

Wired but not yet turning — and why

Loop	State	Why
1 observation	0 live timeline entries	Newest entry predates the deploy. No agent has been executed since. Needs traffic, not repair.
1 drift	0 anomalies	1,170 of 1,245 timeline entries are at <code>persona_version = 1</code> , which the worker skips by design. Historical eval-run entries written before the stamping fix. Only new live traffic is eligible.
1 snapshots	Still 1, newest February	<code>create_snapshot</code> fires on a consolidation cycle; none has run since deploy.
2	Queue empty	Follows from Loop 1 observation. The mechanism is closed; nothing has reached it.
4	0 proposals	127 workspaces, none of which has a composition identity — no mission, no strategist — so there is nothing to version. This is an onboarding gap, not a loop defect.

One claim from this work remains unobserved in production: **1,263 episodes are stamped with a persona version and 1,999 are not**, but the newest episode predates the deploying boot, so the stamping fix has not yet been seen to fire on the live execution path.

Loop 5a — was **WIRING BROKEN** ; the verdict was correct, and it is now sound

The mechanism probe reports two HIGH violations. Neither is a code defect; both are data, and the panel is right to refuse to certify the score while they stand.

L5-M04 — 6 roster entries naming no agent, all from a single forecast. The London 32 °C question. Its `agents_used` entries are named `weather_oracle_synoptic_pattern_august_2025`, `macro_forecaster_climate_trend_adjustment`, and so on — `<agent>_<driver>` composites. Those are **FPL agent-statement names**, not agent names. `handlers/forecasts.rs` states the expected shape plainly: `{"name": "macro_data_agent", ...}`. The FPL author named the statements descriptively, attribution joins on that name, and so all five agents on that forecast lose their credit.

Worth knowing it is one bad artefact rather than a systemic six — and it is the same forecast as the climate-routing failure recorded in `67066e4a`, where London 32 °C returned 0.3% against a

13.3% ensemble truth.

L5-M03 — 7 scored forecasts with an empty `agents_used` . Resolved, Brier computed, attributable to nobody. The signal exists and has nowhere to go.

The seam, and how it was closed — 2026-08-16. `agents_used` records *which agent statements ran*, and every calibration reader treats those names as agent identities. Nothing enforced the correspondence, so an FPL author was free to name a statement anything.

Of the two defensible fixes — reject non-resolving statement names at parse time, or resolve once and store the `agent_id` — the second was chosen. Rejecting would break every existing program that names statements after what they compute, which is a reasonable and arguably more readable convention. So `fermi::attribution::roster` resolves at write time: exact match, then longest `<agent>_<suffix>` prefix with a required underscore boundary. Longest and bounded because `macro` must never claim

`macro_forecaster_climate_trend_adjustment` — attributing one agent's Brier score to another is worse than the orphan it replaces, so an unresolvable name is logged at creation and left alone rather than guessed. mig-209 repaired the backlog: 40 statement names, each to the correct agent.

This also cleared **L5-M09**, which existed to warn that the write path emitted no `agent_id` and would "grow with every new forecast until the write path stamps `agent_id` at creation".

L5-M03 was narrowed, on evidence. It counted every scored forecast with no resolvable roster, conflating *a roster that should exist and doesn't* with *a forecast that legitimately had no agents*. Of 48 resolved-and-scored forecasts whose `fpl_source` declares an `agent` statement, **zero** have an empty roster; of the 7 declaring none, 5 do. The correlation is total. Those five are programs of static drivers with no agent statement — one titled `v0.10.12 sanity check` — and they were failing the entire fleet verdict while behaving correctly.

The check now requires the FPL to have declared an agent, with that evidence in the comment in both copies so the narrowing is auditable rather than looking like score-massaging. **Nine of nine mechanisms now report OK: `MECHANISM SOUND`** .

Narrowing a HIGH-severity check to make a dashboard green is exactly the move that deserves scrutiny, which is why the evidence is recorded rather than the conclusion.

Read the skill score, not the raw one. The same panel reports `99% raw · n=48 · skill +0.35 vs 2% base rate` . A 2% base rate is exactly the outcome-skewed case `compute_agent_calibration` 's own doc-comment warns about: 99% raw is uninformative, **+0.35 skill is the real number**, and it is good.

The zero-entity extraction — resolved 2026-08-16

A dreaming cycle on `football_analyst` reported 12 episodes → 0 entities, 0 facts, 5 rules . Not a silent extractor failure: **the extractors were never shown the answers.**

Both `extract_entities_with_llm` and `extract_knowledge_rules` built their prompts from `Query` plus a truncated `Context` preview. `response_text` appeared in `consolidation.rs` only in test fixtures. mig-199 added the column precisely so this would be available, and nothing then read it — the same shape as every other defect in this document.

It is backwards for entity extraction especially. A question names few entities; the answer is where they are. Measured: queries average 487 chars, responses 3,645. One episode asked `"Will Arsenal beat Manchester City in their next Premier League match?"` and answered `"Arsenal won the 2024-25 Premier League title with 85 points"` — an entity and a fact, in the half being discarded. `context` , meanwhile, is execution telemetry: stop reason, token counts, evidence ids. Mostly noise for extraction.

Both extractors now share `episode_digest` , which includes the response at a 1,200-char budget — twenty unabridged 5k responses would be ~25k tokens of prompt against a 2,048-token completion. The entity prompt now states that the Response is the primary source, because the previous wording described the input as "execution logs" while the digest carried only the query.

This does not retroactively help the 3,298 episodes with no stored response. Retention began with mig-199; 71 episodes have one today. It changes what every cycle from here learns, which is the honest scope.

6. Loop instrumentation

The 2026-06-03 revision had no notion of measuring the loops themselves. Four instruments now exist, and they should be consulted before any claim that a loop is working — each derives independently of the code path it assesses, which is what makes it worth trusting:

Instrument	What it answers	Where
GET /api/me/loop-health	Live per-loop health aggregation, Loops 1–5	src/api_server.rs → handlers::agents::loop
GET /api/observatory/loops/dreaming/maturity	Is Loop 1 running-but-learning-nothing? (the "91 cycles, zero rules" mode)	src/handlers/dreaming_
Observatory Loops tab (/observatory? agent=<name>)	Per-agent RSI loop health, each row derived from a named query and labelled closed / partial / open / broken / unmeasured . Distinguishes <i>thin</i> (sound wiring, little data — wait) from <i>broken</i> (faulty wiring — repair), because the remedies are opposite	src/handlers/observato templates/observatory.

Instrument	What it answers	Where
<code>agent_evolution</code> ledger	Four un-averaged progression dimensions — <code>memory</code> (Loop 1), <code>judgment</code> (Loop 5), <code>conduct</code> (Loop 2), <code>craft</code> — with a <code>peak_level</code> ratchet so regression is measurable	<code>migrations/190_agent_e</code> <code>src/handlers/evolution</code>

The dimensions are deliberately not averaged into a single score, and `agent_evolution` deliberately replaced an activity-based maturity metric that was measuring nothing but usage.

Diagnostic scripts, all read-only and all safe to run against production through `scripts/run_loop5_probe.sh` (which forces a direct connection and statically refuses to run a file containing mutating statements):

Script	Answers
<code>loop1_retrievability_census.sql</code>	Can each agent actually recall what it learned? Grades every agent <code>OPEN</code> / <code>UNEMBEDDED</code> / <code>EMPTY</code> , and separates CEP seeds (always injected) from stranded rows
<code>loop1_maturity_census.sql</code>	Did consolidation produce anything at all?
<code>loop1_extractor_readiness.sql</code>	Is the ontologist's credential funded? Run this before re-dreaming anything
<code>loop5_brier_mechanical_check.sql</code>	Does the Brier chain move a signal correctly? Nine mechanism checks, fleet-wide and per-agent
<code>loop_deploy_check.sql</code>	Post-deploy smoke check

Two are write scripts and are dry-run by default, requiring `-v apply=1` :

`loop1_reset_sterile_episodes.sql` (recover episodes consumed by extractor-less runs, per-episode) and `loop1_dedupe_seed_entities.sql` (collapse duplicate seed rows; `-v reseed=1` deletes all copies so the next boot regenerates them embedded).

7. Open breaks — consolidated

Every verified break, ordered by cost-to-fix against value:

#	Break	Loop	Fix size
1	get_agent_calibration has no dispatch arm; router Stage 0 gets Unknown tool	5a, 5b	Fixed 2026-08-15 — arm + <code>BuiltinToolDef</code> computation extracted to <code>fermi::calibration</code> so route and tool share one implementation
2	propose_composition_change has no dispatch arm	3b, 4	Fixed 2026-08-15 — arm + <code>BuiltinToolDef</code> writes a pending <code>composition_versions</code> row
3	Composition accept path writes <code>teams.member_weights</code>, a column that does not exist	4	Fixed 2026-08-15 — reconciles <code>workspace_agents</code> , transactionally, strategically exempt from eviction
4	Phantom tool regression test covers only 4 weather agents	all	Fixed 2026-08-15 — <code>no_curated_card</code> declares a phantom tool scans all of <code>agents/curated</code> as a ratchet : if pre-existing declarations are quarantined in <code>known_debt</code> , anything new fails, and the list may only shrink
5	Live executions write no eval signal / timeline entry, so drift and anomaly detection never see real traffic	1, 2	Fixed 2026-08-15 — <code>handlers::live_observability</code> : deterministic evaluators only, fire-and-forget, plus a scan sweeper
6	<code>ConsolidationWorker</code> never reads <code>authority_weight</code>, and synthetic corrections are written unembedded, so a human correction can neither cluster nor survive the extraction budget	2	Fixed 2026-08-15 — <code>with_embedder</code> + <code>rank_success_episodes_by_authority</code> ; locked by <code>consolidation::authority_test</code>
7	Coherence shelf executes the strategist without a <code>ToolContext</code>; Stages 0 and 3 are inert	3a	Fixed 2026-08-15 — routed through <code>ToolAwareExecutor</code> with a full <code>ToolContext</code>

#	Break	Loop	Fix size
8	<code>_coordination/brief.md</code> sits outside the <code>context/</code> prefix	3a	Superseded 2026-08-15 — the brief was never the mechanism; <code>record_coordination_observation</code> writes into member memory instead. The brief remains, for humans
9	<code>create_snapshot</code> reachable only from the CLI	1	Fixed 2026-08-15 — see 9d

| 9a | `get_ontology` read `ontology_snapshots` (never written on the API path) and hardcoded empty entity/relationship arrays, so a successful dreaming cycle displayed as zero | 1 | **Fixed 2026-08-15** — reads live tables; locked by `handlers::ontology::tests` || 9c | KG injection gated on `card.ontology_stats`, which nothing maintains (sole updater queried the nonexistent table `kg_entities`), so learned knowledge was never retrieved into any execution | 1 — **was the loop's actual break** | **Fixed 2026-08-15** — gate queries the knowledge tables; locked by `kg_context::gate_tests` || 9d | `create_snapshot` never called on the API path, so ontologies never developed and the narrator's synopsis write was a no-op | 1 | **Fixed 2026-08-15** — `snapshot_ontology` on the dreaming path, push disabled, failure non-fatal || 9b | Agent-level episode recovery excludes any agent that has since learned anything, so investigating a damaged agent by re-dreaming it forfeits recovery | 1 | **Addressed** — `scripts/loop1_reset_sterile_episodes.sql` recovers per-episode || 10 | Valence-homophily threshold (`spread < 0.25`) exists only as prompt text | 3b | Compute it, or stop documenting it as a mechanism || 11 | `route_outcomes` joins heuristically on `(agent_id, driver)` within a time window | 5 | Stamp `episode_id` onto the claim row (deliberately deferred) |

The `detected_at` incident — why this section declares columns

Worth recording in full because it is the whole class in miniature, and it happened *after* everything above was fixed.

The Loop 2 row on the observatory panel read `unmeasured`, with column `"detected_at"` does not exist. `anomaly_events` has `created_at` (mig-105); nothing has ever had a `detected_at`.

The dangerous part is the label. `unmeasured` reads as *"no data yet, come back later"* — and Loop 2 is precisely the loop whose queue is legitimately empty at first, which makes the wrong reading the plausible one. A broken query presented as a young loop. Had the panel said `error` it would have been fixed in minutes.

`anomaly_events` was in `SCHEMA_TABLES` but **none of its columns were in `SCHEMA_COLUMNS`**, so the trust contract could not catch it. That is the same gap that let `kg_entities` — a table that has never existed — sit in a query for months. Declaring the six columns the panel reads means a rename now fails in CI instead of on the dashboard.

The rule this yields: declaring the columns a loop's *reporting* query depends on is part of closing that loop. An instrument that cannot fail loudly is not an instrument.

Open breaks, 2026-08-16

#	Break	Loop	Notes
12	<code>fermi_forecasts.agents_used</code> records FPL statement names, which calibration readers treat as agent identities	5a	Fixed 2026-08-16 — <code>fermi::attribution::roster</code> resolves at write time; mig-209 repaired 40 entries. L5-M03 narrowed on evidence. Probe now reports MECHANISM SOUND, 9/9
13	Entity extraction returned 0 entities on 12 episodes while rule extraction returned 5	1	Fixed 2026-08-16 — neither extractor read <code>response_text</code> ; both now share <code>episode_digest</code> . Not a failure, an omission
14	127 workspaces have no composition identity, so Loop 4 has nothing to version	4	Open. Onboarding gap rather than loop defect
15	73 curated tool declarations remain undispatchable, quarantined in <code>known_debt</code>	all	Ratcheting down: 92 → 79 → 73. Breakdown below
10	Valence-homophily threshold (spread < 0.25) exists only as prompt text	3b	Compute it, or stop documenting it as a mechanism
11	<code>route_outcomes</code> joins heuristically on <code>(agent_id, driver)</code> within a time window	5	Stamp <code>episode_id</code> onto the claim row (deliberately deferred)

The phantom-tool debt

Breaks 1–4 were the phantom-tool family, all closed. The corpus-wide ratchet that replaced them, `no_curated_card_declares_a_phantom_tool`, started at 92 and is at **73**. It has two assertions and a stale-entry check, so a fixed card *must* leave the list — which is what keeps it ratcheting rather than rotting. It fired correctly three times during this work.

What remains, categorised — the point being that most of it is not loop-related:

Category	Count	Nature
Third-party integrations never built	~42	<code>adaptogen_curator</code> (11), <code>stripe_billing</code> (9), <code>instagram_publisher</code> (6), <code>bluesky_publisher</code> (5), <code>social_media_studio</code> (5)... Cards written against APIs with no backend. Needs implementation or removal — a product decision
Loop-relevant	4	<code>dyad_observer</code> (2: <code>query_episodes</code> , <code>query_persona_history</code>) and <code>dream_coordinator</code> / <code>dream_narrator</code> (2: <code>consolidation_reader</code> , <code>agent_profile_loader</code>). <code>intention_coordinator</code> 's six are done — see below
Fabricated helpers	~27	<code>performance_coach</code> , <code>ar_avatar_renderer</code> , <code>daily_puzzle</code> and others declare plausible-sounding tools that were never real

`intention_coordinator` is fixed — 2026-08-16. All six tools now dispatch, backed by `workspace_intentions` (mig-210) rather than the `_coordination/intention_map.json` the card described: a git file has no concurrency story, and the whole point is several agents declaring at once.

Detection lives in `fermi::intentions` and is explicit about its limits, which matters more here than usual:

Class	Decidable	How
Resource conflict	certainly	two active intentions naming the same target
Dependency	certainly	<code>depends_on</code> naming an output nothing produced
Duplication	probabilistically	$\text{cosine} \geq 0.82$ on embedded descriptions
Contradiction	no	needs to understand the claim

Duplication is embedded **on the write path** at 0.82 — not the 0.30 used for KG retrieval, because a false positive there costs tokens whereas here it tells two agents to stop and differentiate. **Contradiction is not detected**; it is surfaced as a duplication candidate for the

caller to judge and never asserted. Claiming otherwise would be the same error as reporting a number whose wiring had not been checked.

`emit_coherence_signal` posts into the conversation as well as recording, because `ConversationObserver::observe` builds the TEC graph from workspace messages — a row in a table nothing reads would be the deferred-work pattern again. `suggest_differentiation` reports the overlap axes and declines to prescribe the split: it holds two descriptions and no knowledge of the workspace goal, so any division of labour it invented would be a guess dressed as advice.

8. What makes this architecture coherent

Each loop corrects at the appropriate timescale:

- Fast loops (1, 2, inner-3) handle execution-level errors — the agent said the wrong thing, the team went in the wrong direction.
- Slow loops (outer-3, 4) handle structural errors — the team is wrong for the problem, the composition needs to change.
- Calibration loop (5) handles systematic bias — the routing classifier has persistent blind spots that need data to reveal.
- Offline loop (A) handles parameter bias — the distribution assumptions the forecasts run on are not grounded in operational data.

Each loop uses a different corrective mechanism:

- Loops 1 and 2: episodic memory → dreaming → semantic rules
- Loop 3: TEC coherence → coordination observation written into member memory → semantic rule → changed behaviour next execution
- Loop 4: Shapley attribution → composition proposals → human approval → team change
- Loop 5: calibration scores + route provenance → routing weights → member selection
- Loop A: observation history → posterior fit → FPL distribution parameters

Each online loop (1–5) is separated from the others by a human or coherence gate:

- Loop 2 requires a human reviewer (anomaly → HITL queue), and a second reviewer for agent-wide scope
- Loop 4 requires owner approval (composition proposal → accept/reject), and proposals are suppressed below 5 forecasts of evidence

- Loop 5's routing weights are readable by humans via the calibration endpoint, and its mechanism probe refuses to certify a score whose wiring is unsound rather than reporting the number anyway

Loop A is separated from Loop B by the operator: fitted parameters pass a Monte Carlo impact gate and either auto-accept or stage a pending row for review. Parameter changes to forecast models are reviewable before they affect production forecasts.

No online loop can modify agent behaviour without either a human gate or the coherence gate. Loop A cannot modify forecast behaviour without passing the impact gate. These properties compound: the system learns continuously at the harness level (Loops 1–5) while requiring human acceptance of parameter-level changes (Loop A). Fast adaptation where the cost of error is low; human review where the cost is high.

9. A closing note on these revisions

The architecture was sound. What the 2026-06-03 revision got wrong was not the design — it was mistaking a declaration for an implementation, in a system where declarations are cheap and look exactly like implementations from the outside.

Every defect found across both revisions reduces to one shape: **a write path that worked, and a read path pointing somewhere else.**

- Consolidation wrote entities; the UI read `ontology_snapshots` .
- Consolidation embedded rules; retrieval gated on a counter nothing maintained.
- HITL wrote corrections; clustering required an embedding they never got.
- The strategist wrote a brief; dreaming reads episodes.
- The router asked for calibration; the tool had no dispatch arm.
- The panel asked for `detected_at` ; the column is `created_at` .

None of these were visible as errors. Four of them were *documented* as working, and three were protected by a comment asserting some other component would finish the job. That is why the remedies in this document are structural rather than editorial: a ratchet that only shrinks, payload assembly that fails in CI, a schema contract that names the columns a loop depends on, and an instrument that says `broken` where it used to say `unmeasured` .

What is genuinely unfinished is honest to state plainly. Every loop is wired, four have been observed turning on real data, and the two exceptions named in the previous revision are both closed: Loop 5a's attribution now resolves at write time and its probe reports `MECHANISM SOUND` , and `intention_coordinator` 's six tools dispatch, so Loop 3's Stage 0 exists for the first time.

What remains is not loop wiring. 127 workspaces have no composition identity, so Loop 4 has nothing to version — an onboarding gap. 73 curated tool declarations are still undispatchable, four of them loop-relevant, ratcheting down. And most loops are waiting on traffic rather than repair, which is a matter of use, not engineering.

The system now learns continuously at the harness level while requiring human acceptance of parameter-level changes. Fast adaptation where the cost of error is low; human review where the cost is high. Both halves are, for the first time, measurable — which is the only claim in this document that matters.